KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Scholarship Eligibility System

Submitted by

AKSHAI PRANAV GB

25BIT002

DHILEEBAN A

Under the Guidance of

Technical Trainer
Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled **"Scholarship Eligibility System"** submitted in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]** is a record of my original work carried out under the guidance of **[DHILEEBAN A]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026
Coimbatore

AKSHAI PRANAV GB

CERTIFICATE

This is to certify that the project entitled **"Scholarship Eligibility System"** is a bona fide work carried out by **[AKSHAI PRANAV] [I-B.SC. INFORMATION TECHNOLOGY]**, [INFORMATION TECHNOLOGY], KG College of Arts & Science, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period April 2026.

Place: COIMBATORE
Date: 9/04/2026

Submitted for the Viva-Voce Examination held on **09-04-2026**

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	5
	1.1 Objective	5
	1.2 Overview	6
	1.3 Features	6
2	**Program Analysis**	7
	2.1 Problem Definition	7
	2.2 Project Overview	7
	2.3 Scopes and Limitations	8
3	**System Design**	9
	3.1 Modules	9
4	**Methodology**	11
	4.1 Problem Analysis	11
	4.2 System Design	11
	4.3 Program Development	12
	4.4 Testing and Validation	13
	4.5 Result and Documentation	14
	4.6 Algorithm	14
	4.7 Flowchart	16
5	**Source Code**	18
6	**Input and Output Screens**	21
7	**Discussion and Conclusion**	26
	7.1 Discussion	26
	7.2 Conclusion	27
8	**References**	27

1. Introduction

Education is the most powerful tool for personal and social growth, but the cost of higher education is a real obstacle for many deserving students. To reduce this burden, colleges, governments and private institutions offer scholarships that reward academic excellence and support students from economically weaker families. Every year, a large number of students apply, and each application must be checked against a set of rules such as minimum marks, minimum attendance and family income limits before a scholarship can be approved.

In most colleges this verification is still carried out manually. Staff members collect mark statements, calculate averages with a calculator, cross-check attendance registers and compare income certificates against the published criteria. The work is repetitive, slow and prone to human error, and a single wrong calculation can deny a deserving student a scholarship or wrongly award one to an ineligible student.

The **Scholarship Eligibility System** is a menu-driven Java application that automates this process. The user enters a student's register number, name, marks in five subjects, attendance percentage and annual family income. The program validates the data, calculates the percentage and grade, applies the scholarship rules and immediately reports whether the student is eligible, which scholarship category applies and how much fee concession is awarded. All records are kept in memory so that they can be viewed, searched and summarised during the same session.

1.1 Objective

The main objective of this project is to design and develop a simple, accurate and easy-to-use Java application that decides scholarship eligibility for students automatically, using clearly defined academic and financial criteria. The specific objectives are:

- To accept student details (register number, name, five subject marks, attendance and family income) through a simple console menu.
- To validate every input so that wrong values, such as marks above 100, are rejected at entry time.
- To calculate the overall percentage and assign a grade (O, A+, A, B, C or F) automatically.
- To apply the scholarship rules consistently and decide between Merit Scholarship (100%), Merit Scholarship (50%), Need-Based Scholarship (25%) and Not Eligible.
- To calculate the exact scholarship amount awarded against the annual tuition fee.
- To store all processed records and allow viewing, searching by register number and summary reporting.
- To apply core Java concepts such as classes, objects, encapsulation, arrays, collections, loops and conditional statements in a practical project.

1.2 Overview

The system is developed in Java using an object-oriented approach. It consists of two classes. The **Student** class represents one student; it holds the personal and academic data as private members and contains the logic that computes the percentage, the grade and the scholarship result when an object is created. The **ScholarshipSystem** class contains the `main()` method, the menu and the supporting methods for adding, viewing, searching and summarising records. Student objects are stored in an `ArrayList<Student>`, which grows automatically as new records are added.

The program runs in a loop until the user chooses to exit. In every iteration the main menu is displayed, the user's choice is read through the `Scanner` class and a `switch` statement transfers control to the correct method. Because the eligibility rules are placed inside the Student class, the same rules are applied to every student, and a change in policy needs to be made in only one place.

1.3 Features

- **Menu-driven interface:** simple numbered options that anyone can use without training.
- **Input validation:** marks outside the range 0–100 are rejected and the user is asked to re-enter the value; invalid menu choices are reported.
- **Automatic percentage and grade calculation:** the average of five subjects is converted into a grade from O to F.
- **Rule-based eligibility check:** minimum attendance, minimum pass mark in every subject, merit criteria and income-based criteria are all applied automatically.
- **Scholarship amount calculation:** 100%, 50% or 25% of the annual tuition fee is awarded as per the category.
- **Record management:** all processed students are stored and can be displayed together at any time.
- **Search facility:** a student's complete result can be retrieved using the register number.
- **Summary report:** total students, eligible students, non-eligible students and the total scholarship amount awarded.
- **Fast and error-free:** removes manual calculation and ensures that the same rules are applied to every student.

2. Program Analysis

2.1 Problem Definition

Scholarship departments must examine hundreds of applications within a limited time. The conventional manual procedure has the following problems:

- **Calculation errors:** averages, percentages and grades are calculated by hand, so mistakes are common, especially during peak periods.
- **Inconsistent decisions:** different staff members may interpret the criteria differently, leading to unfair results.
- **Time consumption:** verifying marks, attendance and income for each applicant takes a great deal of time.
- **Poor record keeping:** paper files are difficult to search, and producing a summary of how many students received scholarships and how much money was awarded is a slow process.
- **Lack of transparency:** students do not clearly know why they were declared eligible or not eligible.

The problem, therefore, is to build a software solution that accepts a student's academic and financial details, applies the scholarship criteria in a uniform way and produces an accurate, immediate and explainable eligibility decision along with the scholarship amount.

2.2 Project Overview

The Scholarship Eligibility System receives the details of a student, processes them with the help of the eligibility rules and generates the result. The overall input, process and output of the system are summarised below.

Stage	Details
Input	Register number, student name, marks in five subjects (0–100 each), attendance percentage, annual family income (Rs.)
Processing	Validate marks; compute total and percentage; assign grade; check minimum pass marks and attendance; apply merit and need-based rules; compute scholarship amount
Output	Percentage, grade, attendance, family income, scholarship category and scholarship amount; list of all students; search result; summary report

The grades are assigned on the basis of the overall percentage as follows:

Percentage	Grade	Percentage	Grade
90 and above	O (Outstanding)	60 to 69.99	B
80 to 89.99	A+	50 to 59.99	C
70 to 79.99	A	Below 50	F (Fail)

The scholarship criteria used by the system are given below. The annual tuition fee is taken as Rs.50,000 and the family income limit for need-based assistance as Rs.2,50,000; both are constants in the program and can be changed easily.

Category	Percentage	Attendance	Other condition	Award
Merit Scholarship	90% or above	85% or above	Pass in all subjects	100% (Rs.50,000)
Merit Scholarship	80% to below 90%	80% or above	Pass in all subjects	50% (Rs.25,000)
Need-Based Scholarship	60% or above	75% or above	Family income up to Rs.2,50,000	25% (Rs.12,500)
Not Eligible	Any other case, any mark below 40, or attendance below 75%			Nil

Software and hardware requirements

Item	Requirement
Programming language	Java (JDK 8 or higher)
Concepts used	Classes and objects, encapsulation, arrays, ArrayList, Scanner, loops, switch, methods, printf
Editor / IDE	Any Java IDE or editor (NetBeans, Eclipse, IntelliJ IDEA, VS Code) or a terminal
Operating system	Windows, Linux or macOS
Hardware	Any computer with at least 2 GB RAM and a processor capable of running a JVM

2.3 Scopes and Limitations

Scope

- Can be used by a college scholarship cell or an examination office to quickly shortlist students for scholarships.
- Suitable for any course that has five subjects per semester; the number of subjects can be increased by changing a single constant.
- Demonstrates the practical use of object-oriented programming and collections and can be used as a base for larger projects.
- The rules (percentage cut-offs, attendance limits, income limit and fee) can be modified to match the policy of any institution or scholarship scheme.

Limitations

- The application is console based and has no graphical user interface.
- Records are stored in memory (ArrayList) and are lost when the program is closed; no database or file storage is used.

- The system is for a single user and does not have a login or role-based access.
- Documents such as mark sheets and income certificates are not verified by the system; the data entered is assumed to be correct.
- Duplicate register numbers are not blocked; the first matching record is returned by the search option.
- Only the three scholarship categories described above are supported.

3. System Design

3.1 Modules

The project is divided into small modules, each of which performs one clear task. This modular design makes the program easier to write, test and modify.

Module	Implemented by	Description
Student Data Module	Student class (fields, constructor)	Stores the register number, name, marks, attendance and family income as private data members and initialises them through the constructor.
Input & Validation Module	addStudent()	Reads details through Scanner; the do-while loop rejects marks below 0 or above 100 and asks the user to enter the value again.
Calculation Module	Student.calculate()	Totals the marks, computes the percentage and assigns the grade O, A+, A, B, C or F.
Eligibility Module	Student.calculate()	Checks minimum pass marks, attendance, merit conditions and income to decide the scholarship category and the amount.
Record Management Module	ArrayList<Student> list	Holds every processed student object; grows dynamically.
Display Module	Student.display(), viewAll()	Prints the complete result of one student or of all students in a formatted way.
Search Module	search()	Finds a student by register number (case-insensitive) using a loop and equalsIgnoreCase().
Report Module	summary()	Counts the total, eligible and not-eligible students and the total scholarship amount.
Menu / Controller Module	main()	Displays the menu, reads the choice and calls the correct method using switch inside a do-while loop.

Data design – attributes of the Student class

Attribute	Data type	Purpose
regNo	String	Unique register number of the student (for example 25BIT002)
name	String	Name of the student
marks[5]	int array	Marks obtained in the five subjects
attendance	double	Attendance percentage
income	double	Annual family income in rupees
percentage	double	Calculated average of the five marks
grade	String	Grade derived from the percentage
scholarship	String	Scholarship category or “Not Eligible”
amount	int	Scholarship amount awarded in rupees

Methods of the application

Method	Class	Function
Student(...)	Student	Constructor; stores the data and calls calculate()
calculate()	Student	Private method that finds percentage, grade, scholarship and amount
getRegNo(), getName(), getPercentage(), getGrade(), getScholarship(), getAmount(), getAttendance()	Student	Getter methods that give controlled access to the private data
display()	Student	Prints the formatted result of the student
addStudent()	ScholarshipSystem	Accepts and validates data, creates a Student object and adds it to the list
viewAll()	ScholarshipSystem	Displays every stored record
search()	ScholarshipSystem	Searches a record by register number
summary()	ScholarshipSystem	Prints the summary report
main()	ScholarshipSystem	Entry point; shows the menu and drives the program

4. Methodology

The project was developed by following the stages of a simple software development life cycle: analysis of the problem, design of the solution, coding, testing and documentation. Each stage is described below.

4.1 Problem Analysis

The first step was to understand how scholarship eligibility is decided in a college and to identify the data that is required. It was found that three factors normally decide a scholarship: **academic performance** (marks and grade), **discipline** (attendance) and **financial need** (family income). A student who has failed in any subject is generally not considered. These factors were converted into precise rules that a program can evaluate.

Functional requirements

- The system shall accept the register number, name, five marks, attendance and family income of a student.
- The system shall reject marks that are not between 0 and 100.
- The system shall calculate the percentage and the grade.
- The system shall decide the scholarship category and amount according to the rules.
- The system shall allow all records to be viewed, a record to be searched and a summary to be generated.

Non-functional requirements

- **Accuracy:** the result must always follow the stated rules.
- **Usability:** simple prompts and clearly formatted output.
- **Performance:** the result must be displayed immediately after the data is entered.
- **Maintainability:** rules should be located in one place so that they are easy to change.
- **Portability:** the program should run on any system that has a JVM.

4.2 System Design

The system follows a simple three-part structure: **input**, **processing** and **output**. The menu and the Scanner class form the input part; the Student class with its calculation and eligibility logic forms the processing part; and the display, search and summary methods form the output part. Student objects are kept in an ArrayList, which acts as the in-memory data store.

Part	Components	Responsibility
Input	Menu, Scanner, addStudent()	Collect and validate data from the user
Processing	Student class, calculate()	Compute percentage, grade and eligibility
Storage	ArrayList<Student>	Keep all processed records during the session
Output	display(), viewAll(), search(), summary()	Show results and reports to the user

Object-oriented design. The data of a student is declared *private* and is accessed only through methods, which is an example of **encapsulation**. The calculation is performed inside the constructor so that a Student object is always created in a complete and consistent state. The controller class contains only static methods because the whole application is a single, menu-driven program. **Data flow:** the user enters the details; the controller validates them and creates a Student object; the object computes the result; the object is stored in the list; the result is displayed; the control returns to the menu.

4.3 Program Development

The program was written in Java and was developed in small steps: first the Student class with its calculation logic, then the input and validation method, then the display, search and summary methods, and finally the menu that ties them together. After each step the program was compiled and run to make sure that it worked before proceeding. The Java concepts used in the project are listed below.

Concept	Where it is used
Classes and objects	Student objects created for every record
Encapsulation	Private fields with public getter methods
Constructor	Student(...) initialises the data and triggers the calculation
Arrays	int[] marks for the five subject marks
Collections	ArrayList<Student> to store the records
Scanner	Reading integers, decimals, words and lines from the keyboard
Loops	for loop (marks), enhanced for loop (list), do-while loop (validation and menu)
Decision making	if-else-if ladder for grade and scholarship; switch for the menu
Formatted output	printf with %.2f and %n for tidy percentages

4.4 Testing and Validation

The program was tested to check that every rule behaves correctly, including the boundary values. **Unit-level testing** was done on the calculation logic; **functional testing** was done on every menu option; **validation testing** was done with wrong inputs such as marks above 100 and an invalid menu choice. The table below lists the test cases. The "Actual Result" column was obtained by running the program.

ID	Input	Expected Result	Actual Result	Status
T1	Marks: 95, 92, 96, 91, 94 | Att: 92% | Income: Rs.180000	Merit Scholarship - 100%	Merit Scholarship - 100% (93.60%, O)	Pass
T2	Marks: 82, 78, 85, 80, 81 | Att: 88% | Income: Rs.400000	Merit Scholarship - 50%	Merit Scholarship - 50% (81.20%, A+)	Pass
T3	Marks: 65, 70, 62, 68, 60 | Att: 90% | Income: Rs.200000	Need-Based Scholarship - 25%	Need-Based Scholarship - 25% (65.00%, B)	Pass
T4	Marks: 72, 35, 66, 58, 61 | Att: 90% | Income: Rs.150000	Not Eligible	Not Eligible (58.40%, C)	Pass
T5	Marks: 90, 92, 95, 35, 96 | Att: 92% | Income: Rs.180000	Not Eligible	Not Eligible (81.60%, A+)	Pass
T6	Marks: 85, 85, 85, 85, 85 | Att: 70% | Income: Rs.180000	Not Eligible	Not Eligible (85.00%, A+)	Pass
T7	Marks: 90, 90, 90, 90, 90 | Att: 84% | Income: Rs.500000	Merit Scholarship - 50%	Merit Scholarship - 50% (90.00%, O)	Pass
T8	Marks: 65, 65, 65, 65, 65 | Att: 90% | Income: Rs.300000	Not Eligible	Not Eligible (65.00%, B)	Pass
T9	Marks: 60, 60, 60, 60, 60 | Att: 75% | Income: Rs.250000	Need-Based Scholarship - 25%	Need-Based Scholarship - 25% (60.00%, B)	Pass

Validation tests

Test	Input	Expected behaviour	Status
Marks above 100	105 for subject 1	Message "Invalid marks! Try again." and the prompt is repeated	Pass
Invalid menu option	9	Message "Invalid choice!" and the menu is shown again	Pass
Search for a missing record	Unknown register number	Message "Student not found."	Pass
View with no records	Option 2 before adding students	Message "No records available."	Pass

4.5 Result and Documentation

All the test cases produced the expected results. The system correctly separated the students into the three scholarship categories and the not-eligible group, and the boundary cases (exactly 75% attendance with Rs.2,50,000 income, and 90% marks with 84% attendance) were handled as designed. For the four sample students used in the output screens, the results are summarised below.

Reg. No.	Name	Percentage	Grade	Result	Amount (Rs.)
25BIT002	Akshai Pranav GB	93.60	O	Merit Scholarship - 100%	50,000
25BIT015	Rahul K	81.20	A+	Merit Scholarship - 50%	25,000
25BIT021	Meena S	65.00	B	Need-Based Scholarship - 25%	12,500
25BIT030	Karthik R	58.40	C	Not Eligible	0

The summary report for these four students shows 3 eligible and 1 not eligible student, with a total of Rs.87,500 awarded. The project is documented through this report, which contains the problem definition, design, algorithm, flowcharts, complete source code and the screenshots of the program output. The source code itself is organised with meaningful names for classes, methods and variables so that it is easy to read and maintain.

4.6 Algorithm

Main algorithm

Step 1: Start.

Step 2: Create an empty list to store Student objects.

Step 3: Display the menu: 1. Add Student, 2. View All, 3. Search, 4. Summary, 5. Exit.

Step 4: Read the choice of the user.

Step 5: If the choice is 1, go to the Add Student algorithm. If 2, display all records (or “No records available” if the list is empty). If 3, read a register number and search the list. If 4, generate the summary report. If 5, print “Thank you!” and go to Step 7. Otherwise print “Invalid choice!”.

Step 6: Go back to Step 3.

Step 7: Stop.

Add Student and eligibility algorithm

Step 1: Read the register number and the name of the student.

Step 2: For i = 1 to 5, read the marks of subject i. If the marks are less than 0 or greater than 100, display “Invalid marks! Try again.” and read again.

Step 3: Read the attendance percentage and the annual family income.

Step 4: Compute total = sum of the five marks and percentage = total / 5.

Step 5: Assign the grade: percentage >= 90 → O; >= 80 → A+; >= 70 → A; >= 60 → B; >= 50 → C; otherwise F.

Step 6: Set fee = 50000. Set failed = true if any subject mark is less than 40.

Step 7: If failed is true or attendance < 75, the result is “Not Eligible” and the amount is 0.

Step 8: Else if percentage >= 90 and attendance >= 85, the result is “Merit Scholarship - 100%” and amount = fee.

Step 9: Else if percentage >= 80 and attendance >= 80, the result is “Merit Scholarship - 50%” and amount = fee / 2.

Step 10: Else if percentage >= 60 and income <= 250000, the result is “Need-Based Scholarship - 25%” and amount = fee / 4.

Step 11: Otherwise the result is “Not Eligible” and the amount is 0.

Step 12: Create the Student object, add it to the list and display the result.

Summary report algorithm

Step 1: Set eligible = 0, sum = 0 and total = number of students in the list.

Step 2: For every student, if the result is not “Not Eligible”, increase eligible by 1 and add the amount to sum.

Step 3: Display total, eligible, (total – eligible) and sum.

4.7 Flowchart

Figure 1 shows the overall working of the program, and Figure 2 shows the decision logic used to find the scholarship eligibility of a student.

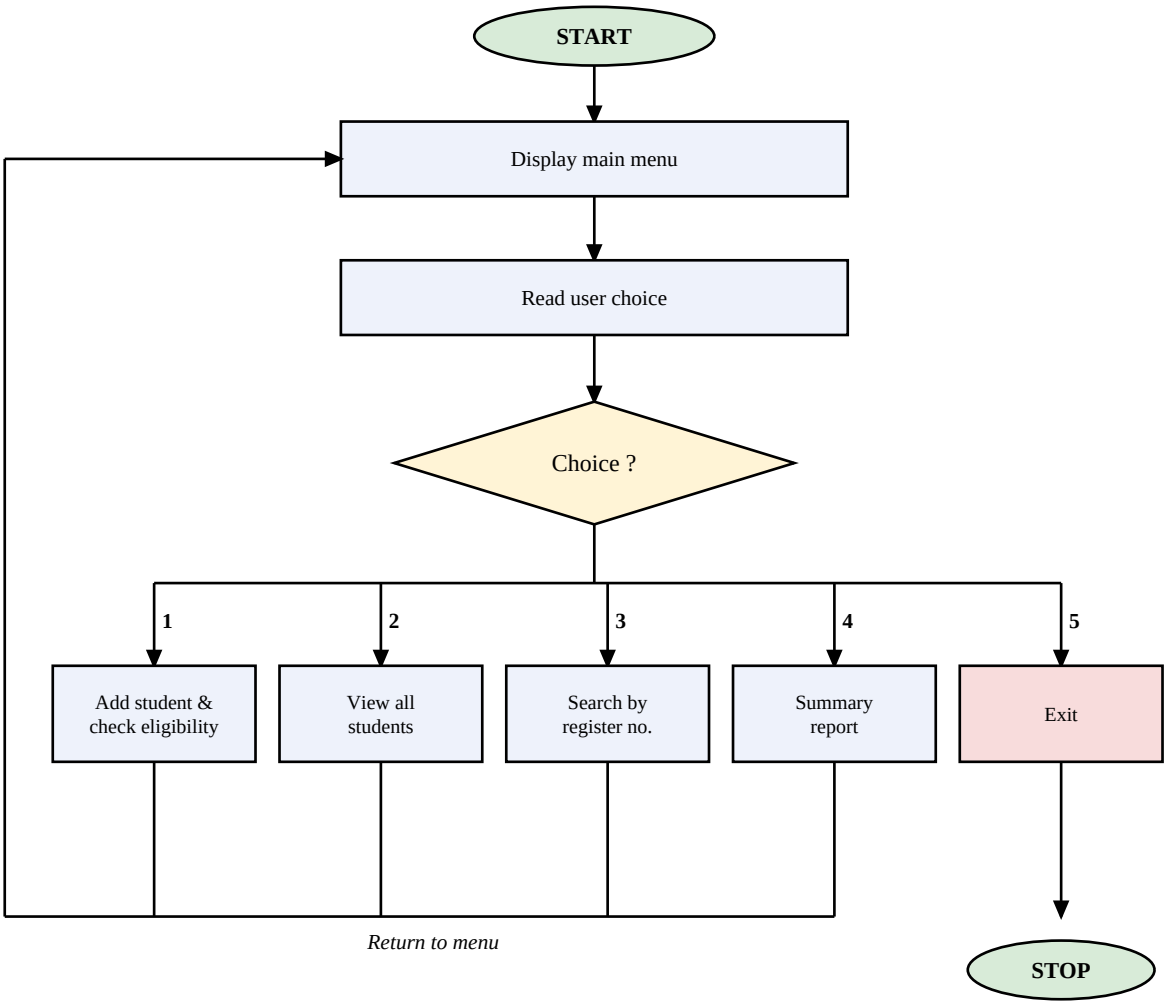


Figure 1: Flowchart of the main program (menu loop)

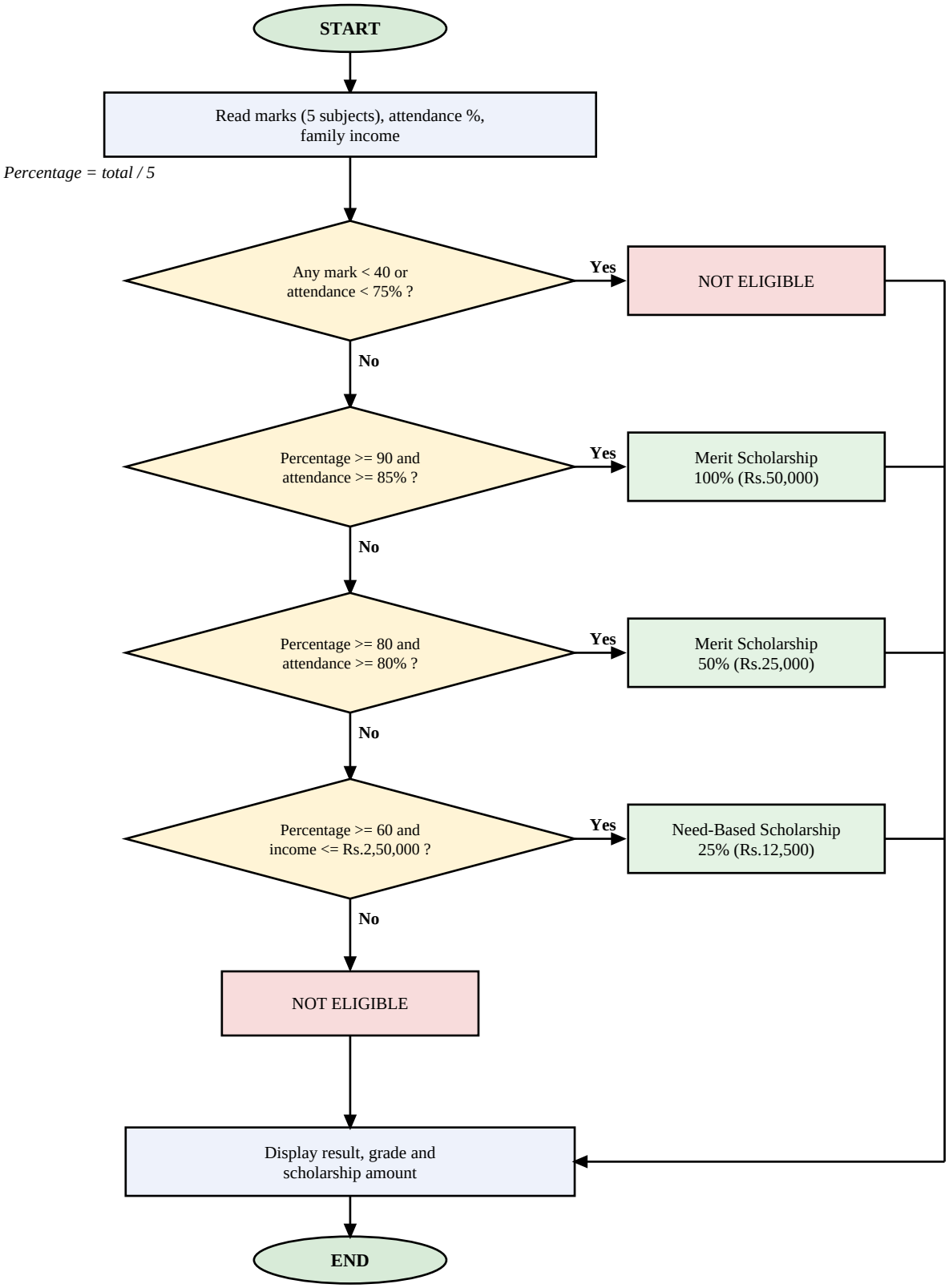


Figure 2: Flowchart of the scholarship eligibility decision

5. Source Code

The complete program is given below. It is saved as `ScholarshipSystem.java`, compiled with `javac ScholarshipSystem.java` and run with `java ScholarshipSystem`.

```
import java.util.ArrayList;
import java.util.Scanner;

public class ScholarshipSystem {
    static ArrayList<Student> list = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);

    static void addStudent() {
        System.out.print("Enter Register Number : ");
        String reg = sc.next();
        sc.nextLine();
        System.out.print("Enter Student Name    : ");
        String name = sc.nextLine();

        int[] marks = new int[5];
        for (int i = 0; i < 5; i++) {
            do {
                System.out.print("Enter marks of subject " + (i + 1) + " (0-100): ");
                marks[i] = sc.nextInt();
                if (marks[i] < 0 || marks[i] > 100)
                    System.out.println("Invalid marks! Try again.");
            } while (marks[i] < 0 || marks[i] > 100);
        }

        System.out.print("Enter Attendance %    : ");
        double att = sc.nextDouble();
        System.out.print("Enter Family Income   : ");
        double inc = sc.nextDouble();

        Student s = new Student(reg, name, marks, att, inc);
        list.add(s);
        System.out.println("\nStudent record added successfully.");
        s.display();
    }

    static void viewAll() {
        if (list.isEmpty()) {
            System.out.println("No records available.");
            return;
        }
        for (Student s : list) s.display();
    }

    static void search() {
        System.out.print("Enter Register Number to search: ");
        String reg = sc.next();
        boolean found = false;
        for (Student s : list) {
            if (s.getRegNo().equalsIgnoreCase(reg)) {
                s.display();
                found = true;
                break;
            }
        }
        if (!found) System.out.println("Student not found.");
    }

    static void summary() {
        int eligible = 0, total = list.size(), sum = 0;
        for (Student s : list) {
            if (!s.getScholarship().equals("Not Eligible")) {
                eligible++;
                sum += s.getAmount();
            }
        }
```

```
        System.out.println("===== SUMMARY REPORT =====");
        System.out.println("Total Students     : " + total);
        System.out.println("Eligible Students  : " + eligible);
        System.out.println("Not Eligible       : " + (total - eligible));
        System.out.println("Total Amount Awarded: Rs." + sum);
    }

    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n===== SCHOLARSHIP ELIGIBILITY SYSTEM =====");
            System.out.println("1. Add Student & Check Eligibility");
            System.out.println("2. View All Students");
            System.out.println("3. Search Student");
            System.out.println("4. Summary Report");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1: addStudent(); break;
                case 2: viewAll();    break;
                case 3: search();     break;
                case 4: summary();    break;
                case 5: System.out.println("Thank you!"); break;
                default: System.out.println("Invalid choice!");
            }
        } while (choice != 5);
    }
}

class Student {
    private String regNo;
    private String name;
    private int[] marks = new int[5];
    private double attendance;
    private double income;
    private double percentage;
    private String grade;
    private String scholarship;
    private int amount;

    public Student(String regNo, String name, int[] marks,
                   double attendance, double income) {
        this.regNo = regNo;
        this.name = name;
        this.marks = marks;
        this.attendance = attendance;
        this.income = income;
        calculate();
    }

    private void calculate() {
        int total = 0;
        for (int m : marks) total += m;
        percentage = total / 5.0;

        if (percentage >= 90) grade = "O";
        else if (percentage >= 80) grade = "A+";
        else if (percentage >= 70) grade = "A";
        else if (percentage >= 60) grade = "B";
        else if (percentage >= 50) grade = "C";
        else grade = "F";

        int fee = 50000;
        boolean failed = false;
        for (int m : marks) if (m < 40) failed = true;

        if (failed || attendance < 75) {
            scholarship = "Not Eligible";
            amount = 0;
        } else if (percentage >= 90 && attendance >= 85) {
            scholarship = "Merit Scholarship - 100%";
```

```
            amount = fee;
        } else if (percentage >= 80 && attendance >= 80) {
            scholarship = "Merit Scholarship - 50%";
            amount = fee / 2;
        } else if (percentage >= 60 && income <= 250000) {
            scholarship = "Need-Based Scholarship - 25%";
            amount = fee / 4;
        } else {
            scholarship = "Not Eligible";
            amount = 0;
        }
    }

    public String getRegNo()       { return regNo; }
    public String getName()        { return name; }
    public double getPercentage()  { return percentage; }
    public String getGrade()       { return grade; }
    public String getScholarship() { return scholarship; }
    public int getAmount()         { return amount; }
    public double getAttendance()  { return attendance; }

    public void display() {
        System.out.println("-------------------------------------");
        System.out.println("Reg No      : " + regNo);
        System.out.println("Name        : " + name);
        System.out.printf("Percentage  : %.2f%%%n", percentage);
        System.out.println("Grade       : " + grade);
        System.out.printf("Attendance  : %.1f%%%n", attendance);
        System.out.printf("Family Income: Rs.%.0f%n", income);
        System.out.println("Result      : " + scholarship);
        System.out.println("Amount      : Rs." + amount);
        System.out.println("-------------------------------------");
    }
}
```

6. Input and Output Screens

The following screens were captured from the running program. Text that follows a prompt is the data typed by the user.

Output Screen 1: Main menu, invalid choice and exit

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 9
Invalid choice!

===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 5
Thank you!
```

Figure 3: Main menu

Output Screen 2: Merit Scholarship (100%)

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 1
Enter Register Number : 25BIT002
Enter Student Name    : Akshai Pranav GB
Enter marks of subject 1 (0-100): 95
Enter marks of subject 2 (0-100): 92
Enter marks of subject 3 (0-100): 96
Enter marks of subject 4 (0-100): 91
Enter marks of subject 5 (0-100): 94
Enter Attendance %    : 92
Enter Family Income   : 180000

Student record added successfully.
-------------------------------------
Reg No      : 25BIT002
Name        : Akshai Pranav GB
Percentage  : 93.60%
Grade       : O
Attendance  : 92.0%
Family Income: Rs.180000
Result      : Merit Scholarship - 100%
Amount      : Rs.50000
-------------------------------------
```

Figure 4: Student with 93.6% marks and 92% attendance

Output Screen 3: Merit Scholarship (50%)

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 1
Enter Register Number : 25BIT015
Enter Student Name    : Rahul K
Enter marks of subject 1 (0-100): 82
Enter marks of subject 2 (0-100): 78
Enter marks of subject 3 (0-100): 85
Enter marks of subject 4 (0-100): 80
Enter marks of subject 5 (0-100): 81
Enter Attendance %    : 88
Enter Family Income   : 400000

Student record added successfully.
-------------------------------------
Reg No      : 25BIT015
Name        : Rahul K
Percentage  : 81.20%
Grade       : A+
Attendance  : 88.0%
Family Income: Rs.400000
Result      : Merit Scholarship - 50%
Amount      : Rs.25000
-------------------------------------
```

Figure 5: Student with 81.2% marks and 88% attendance

Output Screen 4: Need-Based Scholarship (25%)

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 1
Enter Register Number : 25BIT021
Enter Student Name    : Meena S
Enter marks of subject 1 (0-100): 65
Enter marks of subject 2 (0-100): 70
Enter marks of subject 3 (0-100): 62
Enter marks of subject 4 (0-100): 68
Enter marks of subject 5 (0-100): 60
Enter Attendance %    : 90
Enter Family Income   : 200000

Student record added successfully.
-------------------------------------
Reg No      : 25BIT021
Name        : Meena S
Percentage  : 65.00%
Grade       : B
Attendance  : 90.0%
Family Income: Rs.200000
Result      : Need-Based Scholarship - 25%
Amount      : Rs.12500
-------------------------------------
```

Figure 6: Student with 65% marks and family income of Rs.2,00,000

Output Screen 5: Not Eligible

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 1
Enter Register Number : 25BIT030
Enter Student Name    : Karthik R
Enter marks of subject 1 (0-100): 72
Enter marks of subject 2 (0-100): 35
Enter marks of subject 3 (0-100): 66
Enter marks of subject 4 (0-100): 58
Enter marks of subject 5 (0-100): 61
Enter Attendance %    : 90
Enter Family Income   : 150000

Student record added successfully.
-------------------------------------
Reg No      : 25BIT030
Name        : Karthik R
Percentage  : 58.40%
Grade       : C
Attendance  : 90.0%
Family Income: Rs.150000
Result      : Not Eligible
Amount      : Rs.0
-------------------------------------
```

Figure 7: Student who has failed in one subject

Output Screen 6: Input validation of marks

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 1
Enter Register Number : 25BIT040
Enter Student Name    : Divya M
Enter marks of subject 1 (0-100): 105
Invalid marks! Try again.
Enter marks of subject 1 (0-100): 90
Enter marks of subject 2 (0-100): 88
Enter marks of subject 3 (0-100): 92
Enter marks of subject 4 (0-100): 85
Enter marks of subject 5 (0-100): 87
Enter Attendance %    : 91
Enter Family Income   : 220000

Student record added successfully.
-------------------------------------
Reg No      : 25BIT040
Name        : Divya M
Percentage  : 88.40%
Grade       : A+
Attendance  : 91.0%
Family Income: Rs.220000
Result      : Merit Scholarship - 50%
Amount      : Rs.25000
-------------------------------------
```

Figure 8: Marks of 105 are rejected and entered again

Output Screen 7: View all students

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 2
-------------------------------------
Reg No      : 25BIT002
Name        : Akshai Pranav GB
Percentage  : 93.60%
Grade       : O
Attendance  : 92.0%
Family Income: Rs.180000
Result      : Merit Scholarship - 100%
Amount      : Rs.50000
-------------------------------------
-------------------------------------
Reg No      : 25BIT021
Name        : Meena S
Percentage  : 65.00%
Grade       : B
Attendance  : 90.0%
Family Income: Rs.200000
Result      : Need-Based Scholarship - 25%
Amount      : Rs.12500
-------------------------------------
```

Figure 9: Display of all stored records

Output Screen 8: Search by register number

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 3
Enter Register Number to search: 25BIT021
-------------------------------------
Reg No      : 25BIT021
Name        : Meena S
Percentage  : 65.00%
Grade       : B
Attendance  : 90.0%
Family Income: Rs.200000
Result      : Need-Based Scholarship - 25%
Amount      : Rs.12500
-------------------------------------
```

Figure 10: Search result for 25BIT021

Output Screen 9: Summary report

```
===== SCHOLARSHIP ELIGIBILITY SYSTEM =====
1. Add Student & Check Eligibility
2. View All Students
3. Search Student
4. Summary Report
5. Exit
Enter your choice: 4
===== SUMMARY REPORT =====
Total Students     : 2
Eligible Students  : 2
Not Eligible       : 0
Total Amount Awarded: Rs.62500
```

Figure 11: Summary of students and amount awarded

7. Discussion and Conclusion

7.1 Discussion

The Scholarship Eligibility System shows how a real-life decision-making process can be converted into a working program with the help of basic Java features. The most important part of the work was to translate the written scholarship policy into precise conditions. The order of the conditions is very important: the disqualifying checks (failed subject or low attendance) are evaluated first, followed by the merit categories from the highest to the lowest, and finally the need-based category. If the order were changed, a high-scoring student could be given a smaller scholarship by mistake.

Several difficulties were faced during development. Mixing `nextInt()` and `nextLine()` of the Scanner class caused the name to be skipped until the leftover new-line character was consumed. Handling invalid marks required a do-while loop so that the prompt is repeated until a correct value is entered. Testing the boundary values, such as exactly 90% or exactly 75% attendance, helped to confirm that the greater-than-or-equal-to conditions were written correctly.

Through this project the following were learnt: designing a class with private members and getter methods, creating and using objects, storing objects in an ArrayList, building a menu with switch inside a do-while loop, validating user input, formatting output with printf and testing a program systematically with normal and boundary data.

Future enhancements

- Store records permanently in a file or in a database such as MySQL using JDBC.
- Develop a graphical interface using Java Swing or JavaFX, or a web interface.
- Add a login system for administrators and a facility to update or delete records.
- Prevent duplicate register numbers and add exception handling for non-numeric input.
- Support different scholarship schemes, a variable number of subjects and category-based reservations.
- Generate printable reports and export the results to PDF or Excel.
- Send automatic e-mail or SMS notifications to the selected students.

7.2 Conclusion

The Scholarship Eligibility System is an efficient solution that automates the checking of scholarship eligibility. It accepts the academic and financial details of a student, validates them, calculates the percentage and grade, applies the scholarship rules and gives an immediate and accurate decision along with the amount awarded. The system removes manual calculation errors, saves time, applies the same rules to every student and provides a clear summary for the administration.

The project fulfils all the objectives that were set at the beginning, and every test case produced the expected result. It has also given practical experience in object-oriented programming, collections, input validation and testing in Java. With the suggested enhancements, such as database storage and a graphical interface, the system can be developed into a complete scholarship management application for a college.

8. References

1. Software Engineering: A Practitioner's Approach – Concepts of software design and development.
2. Database System Concepts – Fundamentals of database systems.
3. IEEE – Standards and practices for software systems.
4. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill Education – Core Java, classes, collections and the Scanner class.
5. E. Balagurusamy, *Programming with Java: A Primer*, McGraw-Hill Education – Object-oriented programming concepts in Java.
6. Oracle, *Java SE Documentation and API Specification*, docs.oracle.com/javase – ArrayList, Scanner and String classes.
7. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications* – Writing functional and non-functional requirements.